

Structured Output in LangChain

1. What is Structured Output?

- Normally, when you ask a **Language Model (LLM)** something, it replies in **free text** (like an essay).
- Example:
 - You ask: “*Summarize this review.*”
 - It replies: “*The Samsung Galaxy S24 Ultra is fast, powerful, has a great camera, but is heavy and expensive.*”

👉 Problem: If you are building an **app**, free text is **hard to use** in code.

- **Structured Output** means asking the model to reply in a **fixed, machine-readable format** (like JSON, dictionary, or table).
- Example:

json

 Copy code

```
{
  "summary": "Fast, powerful, great camera, but heavy and costly",
  "sentiment": "pos",
  "pros": ["Powerful processor", "200MP camera", "Long battery life"],
  "cons": ["Heavy", "Expensive"]
}
```

👉 Now, your app can **directly use** this output — for API responses, dashboards, or databases.

2. Why Do We Need Structured Output?

- **Free text is messy** → Different outputs each time.
- Apps need **consistent formats**.
- Structured output helps in:
 - **Data extraction**: Pick themes, pros, cons, sentiment.
 - **APIs**: Frontend apps need JSON, not random text.
 - **Agents**: Bots need structured commands to act.

3. Different Ways to Get Structured Output

◆ 3.1 JSON Schema

- **JSON Schema** = A blueprint that defines:
 - What keys should exist
 - What type of values (string, list, number)
 - Which keys are required

✅ Advantage: Works with any programming language (Python, Java, JS).

Example Code:

python

 Copy code

```
json_schema = {
    "title": "Review",
    "type": "object",
    "properties": {
        "summary": {"type": "string"},
        "sentiment": {"type": "string", "enum": ["pos", "neg"]},
        "pros": {"type": ["array", "null"], "items": {"type": "string"}}
    },
    "required": ["summary", "sentiment"]
}

structured_model = model.with_structured_output(json_schema)
result = structured_model.invoke("Review text...")
print(result)
```

◆ 3.2 TypedDict

- **TypedDict** = A Python way to define a dictionary **shape**.
- It says what keys should be there and their types.
- **✗** It does **not** check the data at runtime (only helps during coding).

Example Code:

python

 Copy code

```
from typing import TypedDict, Annotated, Optional, Literal

class Review(TypedDict):
    key_themes: Annotated[list[str], "Themes in review"]
    summary: Annotated[str, "Brief summary"]
    sentiment: Annotated[Literal["pos", "neg"], "Sentiment"]
    pros: Optional[list[str]]
    cons: Optional[list[str]]
    name: Optional[str]
```

👉 Think of it like **filling exam forms**:

- TypedDict says “These fields exist: Name, Age, Marks.”
- But it doesn’t stop you from writing wrong values.

◆ 3.3 Pydantic

- **Pydantic** = A Python library for **data validation**.
- It checks data at runtime and throws errors if invalid.
- You can add:

- **Default values**
- **Constraints** (e.g., CGPA between 0–10)
- **Descriptions**
- **Regex validation**

✓ Advantage: Safer, more reliable than TypedDict.

Example Code:

python

Copy code

```
from pydantic import BaseModel, Field
from typing import Optional, Literal

class Review(BaseModel):
    key_themes: list[str] = Field(description="Themes in review")
    summary: str
    sentiment: Literal["pos", "neg"]
    pros: Optional[list[str]] = None
    cons: Optional[list[str]] = None
    name: Optional[str] = None
```

👉 Think of it like a **teacher checking exam papers**:

- If you write “Age = Hello”, the teacher rejects it.

◆ 3.4 Direct JSON Output

- You can directly ask the LLM:
“Return the result in JSON format.”
- But models sometimes **hallucinate** or make mistakes.
- That’s why LangChain provides `with_structured_output()` to ensure correctness.

4. Code Demonstrations

✓ **Student Example with Pydantic**

python

 Copy code

```
from pydantic import BaseModel, EmailStr, Field
from typing import Optional

class Student(BaseModel):
    name: str = 'nitish'
    age: Optional[int] = None
    email: EmailStr
    cgpa: float = Field(gt=0, lt=10, default=5,
                        description="A decimal value representing CGPA")

new_student = {'age': '32', 'email': 'abc@gmail.com'}
student = Student(**new_student)

print(student.dict())          # Convert to dict
print(student.model_dump_json()) # Convert to JSON
```

👉 Output will always be clean and valid.

✅ TypedDict Person Example

python

 Copy code

```
from typing import TypedDict

class Person(TypedDict):
    name: str
    age: int

new_person: Person = {"name": "nitish", "age": 35}
print(new_person)
```

👉 Here, no validation. If you put "age": "abc", it won't complain.

✅ Review Example with JSON Schema

python

 Copy code

```
structured_model = model.with_structured_output(json_schema)
result = structured_model.invoke("Samsung Galaxy S24 Ultra review...")
print(result)
```

👉 Output will always match the schema (summary, pros, cons, sentiment).

5. When to Use What?

Method	Use Case
TypedDict	For simple projects, type hints only.
Pydantic	For serious apps needing validation & defaults.
JSON Schema	For APIs and cross-language apps.
Direct JSON	For quick experiments, not reliable in production.

6. Key Takeaways

- **Structured output** is the backbone for production AI apps.
 - It makes LLMs predictable and usable.
 - Options available: **TypedDict**, **Pydantic**, **JSON Schema**.
 - LangChain's `.with_structured_output()` makes life easier.
-